

Statische Methoden

Pseudocode (IHK-Stil):

```
funktion quadrat(x: Integer) → Integer
    return x * x
end funktion
```

```
ergebnis = quadrat(4) // 16
```

Java-Beispiel:

```
public class StatischeDemo {
    static int quadrat(int x) { return x * x; }

    public static void main(String[] args) {
        int erg = quadrat(4);
        System.out.println(erg); // 16
        System.out.println(Math.max(3,7)); // 7
    }
}
```

Statische Methoden – Erklärung & Beispiele

Eine statische Methode gehört zur Klasse, nicht zum Objekt.
Aufruf über `Klassenname.methode()`.

****Vorteile:****

- Praktisch für Hilfsfunktionen
- Kein Objekt notwendig

****Nachteile:****

- Kein Zugriff auf Instanzvariablen

****Prüfungstipp:**** Bekannte statische Methoden: `Math.max`, `Math.sqrt`.

Methode	Aufruf	Ergebnis
quadrat(4)	StatischeDemo.quadrat(4)	16
Math.max(3,7)	Math.max(3,7)	7

Methoden mit Objektparametern

Pseudocode (IHK-Stil):

```
klasse Kunde
  attribute: name: String, rabatt: Double

funktion berechneRabatt(k: Kunde) → Double
  wenn k.rabatt > 0 dann
    return k.rabatt
  sonst return 0
end funktion
```

Java-Beispiel:

```
class Kunde { String name; double rabatt; }

public class MethodenMitObjekten {
  static double berechneRabatt(Kunde k) {
    if (k.rabatt > 0) return k.rabatt;
    else return 0;
  }
  public static void main(String[] args) {
    Kunde k = new Kunde();
    k.name = "Anna"; k.rabatt = 0.1;
    System.out.println(berechneRabatt(k)); // 0.1
  }
}
```

Methoden mit Objektparametern – Erklärung & Beispiele

Methoden können ganze Objekte als Parameter erhalten. Dadurch sind komplexe Operationen möglich.

****Beispiel:**** Kunde-Objekt mit Rabattpfeld → Methode berechnet Rabatt. Änderungen wirken auf das gleiche Objekt, da Referenz übergeben wird.

****Typische Fehlerquellen:****

- NullPointerException: wenn null übergeben wird
- Objekt nicht initialisiert

****Prüfungstipp:**** Sehr häufig in IHK: Übergabe von Objekten wie Kunde, Bestellung, Kurs.

Aufruf	Objekt	Ergebnis
berechneRabatt(Kunde)	{rabatt=0.1}	0.1
berechneRabatt(Kunde)	{rabatt=0}	0

Rekursive Methoden (Beispiel Fakultät)

Pseudocode (IHK-Stil):

```
funktion fakultaet(n: Integer) → Integer
    wenn n == 0 dann
        return 1
    sonst
        return n * fakultaet(n-1)
end funktion
```

Java-Beispiel:

```
public class RekursionDemo {
    static int fakultaet(int n) {
        if (n == 0) return 1;
        return n * fakultaet(n-1);
    }
    public static void main(String[] args) {
        System.out.println(fakultaet(5)); // 120
    }
}
```

Rekursive Methoden – Erklärung & Beispiele

Eine rekursive Methode ruft sich selbst auf, bis eine Abbruchbedingung erreicht wird.

Beispiel: Fakultät $n! = n * (n-1)!$

Typische Fehlerquellen:

- Abbruchbedingung vergessen → Endlosschleife
- StackOverflowError bei zu tiefen Aufrufen

Prüfungstipp: Rekursive Methoden werden manchmal bei Algorithmen (Fakultät, Fibonacci) abgefragt.

Eingabe	Ausgabe
fakultaet(0)	1
fakultaet(5)	120

Getter/Setter in Klassen

Pseudocode (IHK-Stil):

```
klasse Kunde
    private name: String

    methode getName() → String
        return name
    methode setName(n: String)
        name = n
end klasse
```

Java-Beispiel:

```
class Kunde {
    private String name;
    public String getName() { return name; }
    public void setName(String n) { this.name = n; }
}
```

Getter/Setter – Erklärung & Beispiele

Getter und Setter dienen der Kapselung (Encapsulation).
Felder sind `private`, Zugriff nur über Methoden.

****Vorteile:****

- Kontrollierter Zugriff auf Felder
- Datenkapselung

****Prüfungstipp:**** Getter/Setter sind Standard bei OOP-Aufgaben.

Methode	Funktion
getName()	Liest den Namen
setName(n)	Setzt den Namen

this / super in Methoden

Pseudocode (IHK-Stil):

```
klasse Tier
    methode sprich()
        ausgabe("Ich bin ein Tier")
```

```
klasse Hund erweitert Tier
    methode sprich()
        super.sprich()
        ausgabe("Wuff!")
end klasse
```

Java-Beispiel:

```
class Tier {
    public void sprich() { System.out.println("Ich bin ein Tier"); }
}
class Hund extends Tier {
    @Override
    public void sprich() {
        super.sprich();
        System.out.println("Wuff!");
    }
}
```

this / super – Erklärung & Beispiele

- `this` → Zugriff auf aktuelles Objekt (z. B. Felder, Konstruktoren).
- `super` → Zugriff auf die Oberklasse (Methoden, Konstruktoren).

****Typische Fehlerquellen:****

- this und super verwechseln
- super-Konstruktor nicht aufrufen

****Prüfungstipp:**** Besonders wichtig bei Vererbung und Konstruktoren.

Schlüsselwort	Funktion
this	aktuelles Objekt
super	Oberklasse